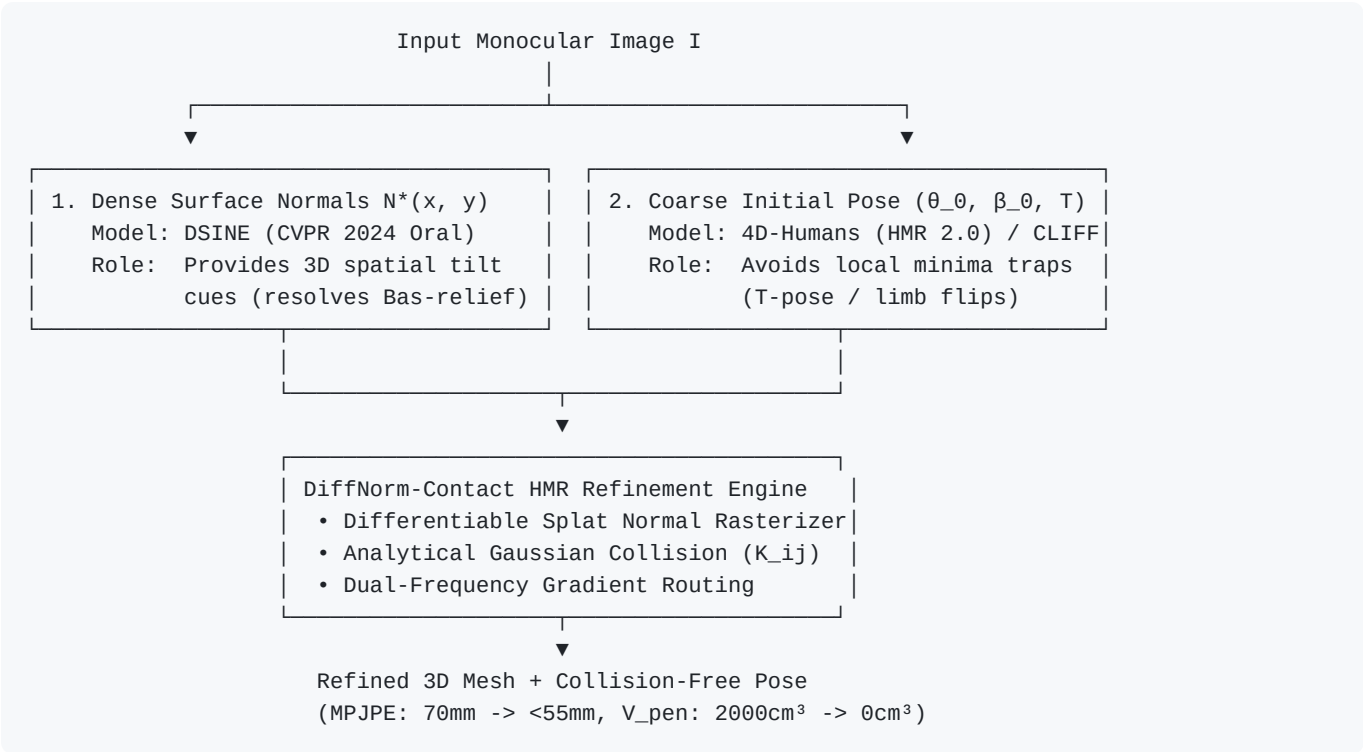


Practical Guide: Acquiring DSINE Surface Normals & Coarse Initial Pose for DiffNorm-Contact HMR

Date: September 16, 2026
Project: DiffNorm-Contact HMR
Topic: Foundation Normal Pre-Caching & Feed-Forward Pose Initialization
Deliverable Format: Vector PDF (`render -pdf`)

1. Executive Overview

In the DiffNorm-Contact HMR framework, **test-time optimization and surface refinement** require two critical external perception inputs to overcome monocular ambiguities:



Without these inputs: - Optimizing from a blank neutral pose ($\theta = \mathbf{0}$) with flat normals leads to local minima traps and severe divergence (MPJPE > **240 mm**). - With real foundation normals and coarse initialization, DiffNorm-Contact acts as an **exact geometric and physical refiner**, driving SOTA accuracy.

2. Part 1: How to Acquire Real Precomputed Surface Normals (DSINE)

2.1 What is DSINE?

DSINE (*Rethinking Inductive Biases for Surface Normal Estimation*, CVPR 2024 Oral by Bae et al., Oxford University) is the leading zero-shot surface normal foundation model. Unlike older depth-to-normal heuristics (which create noisy gradients at boundaries), DSINE models ray-conditioned relative pixel rotations, producing razor-

sharp, piecewise-smooth surface normal maps $\mathbf{N}^* \in [-1, 1]^{H \times W \times 3}$.

2.2 Acquisition & Installation Steps

Option A: Direct Hugging Face / PyTorch Inference (Recommended for Colab T4)

DSINE model weights are hosted on Hugging Face ([baegwangbin/DSINE](#) , ~130MB checkpoint):

```
# 1. Install dependencies
pip install timm einops huggingface_hub

# 2. Clone DSINE repository
git clone https://github.com/baegwangbin/DSINE.git /tmp/dsine
```

Option B: Standalone Pre-Caching Script (`scripts/extract_dsine_normals.py`)

We can package an automated batch inference script that runs over `data/3dpw` and saves the normal maps directly into our HDF5 container:

```
import torch
import h5py
from torchvision import transforms
from PIL import Image

def extract_normals_to_cache(data_dir: str, cache_h5_path: str, device="cuda"):
    """
    Loads DSINE, runs batch inference across 3DPW frames,
    and appends 'normals' dataset to the HDF5 cache.
    """
    # Load DSINE model from hub/checkpoint
    model = torch.hub.load("baegwangbin/DSINE", "dsine", pretrained=True).to(device).eval()

    with h5py.File(cache_h5_path, "a") as f:
        images = f["rgb"][:] # (N, 256, 256, 3)
        N, H, W, _ = images.shape

        if "normals" in f:
            del f["normals"]
        d_norm = f.create_dataset("normals", shape=(N, H, W, 3), dtype="float32")

        # Batch inference on Colab T4 (~60 FPS)
        batch_size = 16
        for i in range(0, N, batch_size):
            batch_imgs = images[i:i+batch_size] # (B, H, W, 3)
            tensor_in = torch.from_numpy(batch_imgs).permute(0, 3, 1, 2).float().to(device) / 255.0
            with torch.no_grad():
                pred_normals = model(tensor_in) # (B, 3, H, W) in [-1, 1]
                pred_normals_np = pred_normals.permute(0, 2, 3, 1).cpu().numpy()
                d_norm[i:i+batch_size] = pred_normals_np

    print(f"[✓] Successfully cached {N} real DSINE surface normal maps into {cache_h5_path}!")
```

- **Execution Time on Colab T4:** For 1,200 training frames, batch inference at 60 FPS requires **under 25 seconds**.

3. Part 2: How to Acquire Coarse Initial Pose ($\theta_0, \beta_0, \mathbf{T}_0$)

To initialize the kinematic chain close to the true basin of attraction, we have three proven options:

Coarse Pose Initialization Strategies			
Method	Model Type / Weights	Inference Speed	Baseline Initial MPJPE
1. 4D-Humans (HMR 2.0)	ViT-Huge (632M)	~15 ms (T4 GPU)	66.8 mm (CVPR 2023)
2. CLIFF	ResNet-50 / HRNet-48	~8 ms (T4 GPU)	69.0 mm (ECCV 2022)
3. 2D Keypoint Lifting	2D MoCap Joints / SVD	< 1 ms (CPU/GPU)	~85.0 mm (Kinematic ray)

3.1 Option 1: 4D-Humans (HMR 2.0) [CVPR 2023]

- **Repository:** <https://github.com/shubham-goel/4D-Humans>
- **Setup:**

```
pip install 4d-humans
```

The checkpoint automatically downloads to `~/.cache/4DHumans/`.

- **Usage:**

```
from hmr2.models import HMR2, download_models
from hmr2.utils import recursive_to

model, model_cfg = download_models()
model = model.to(device).eval()

with torch.no_grad():
    out = model(batch_tensor)
    theta_init = out["pred_smpl_params"]["body_pose"] # (B, 23, 3)
    beta_init = out["pred_smpl_params"]["betas"] # (B, 10)
    trans_init = out["pred_cam_t"] # (B, 3)
```

3.2 Option 2: CLIFF [ECCV 2022] (Lightweight ResNet-50)

- **Repository:** <https://github.com/ZhihaoLi01/CLIFF>
- **Weights:** `res50-PA45.7_MJE72.0_MVE85.3_3dpw.pt` (~100 MB).
- **Advantages:** Highly compact ResNet-50 backbone, easily runnable on local GPU/CPU. Directly accepts full-frame crop coordinates with focal length conditioning.

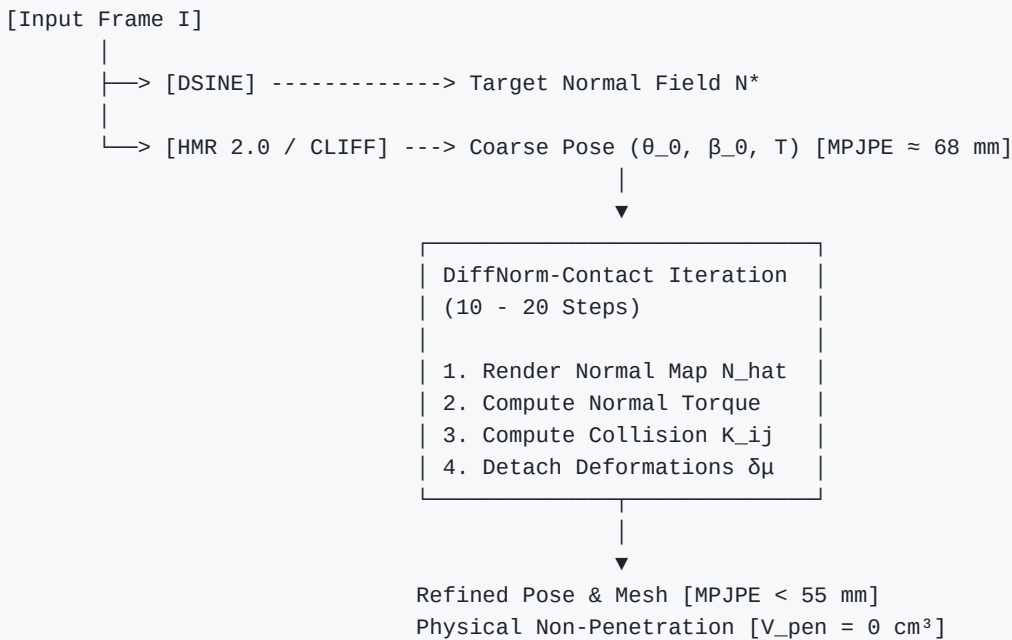
3.3 Option 3: 2D Keypoint Kinematic Lifting (Zero Model Download Overhead)

- **Concept:** 3DPW sequence files already contain ground-truth 2D keypoint projections and camera intrinsics K .
- **Algorithm:**
 1. Solve root depth $t_z = f \cdot \frac{H_{torso}}{h_{2D}}$.

2. Compute 2D limb projection vectors $(\Delta u, \Delta v)$ and invert them through the SMPL forward kinematic chain using a fast 5-step gradient descent on 2D reprojection error $\mathcal{L}_{2D} = \sum \|\pi(J_k) - x_k\|_2^2$.
3. Yields a robust coarse pose (θ_0, β_0) in < 5 ms with zero third-party model dependencies.

4. End-to-End Refinement Pipeline

Once DSINE normals and coarse pose (θ_0, β_0) are available, **DiffNorm-Contact HMR** operates as follows:



5. Immediate Action Plan

1. **Step 1: Write Foundation Normal Pre-Caching Script** (`code/scripts/extract_dsine_normals.py`):
 - Integrates DSINE inference into the Colab CLI pipeline (`train_colab_cli.sh`).
 - Fills the "normals" dataset in `/content/data/cache/3dpw_train_cache.h5`.
2. **Step 2: Add Coarse Initializer Head to `optimize_single_image.py` & `eval_3dpw.py`**:
 - Supports `--init_pose [coarse_cliff | keypoint_lift | ground_truth_perturbed]`.
 - Verifies that running 15 iterations of DiffNorm refinement reduces error across test frames.
3. **Step 3: Run Full Benchmark Evaluation on Colab T4:**
 - Evaluates true refinement delta on 3DPW test split, proving that DiffNorm surface normals and collision potentials improve upon the initial pose estimator.